

PROJECT MANUAL

**TECHNICAL DOCUMENTATION
ASSISTANT**

**USING RETRIEVAL-AUGMENTED
GENERATION (RAG)**

A RAG-Based Chatbot for Querying Technical Documentation

Submitted By

Nekkanti Yamini Vidya

Register Number: 231FA23002

B.Tech – Computer Science and Engineering
(Data Science)

Vignan's Foundation for Science, Technology and Research

Technology Stack

Python Streamlit Sentence Transformers
ChromaDB Google Gemini

GitHub Repository

<https://github.com/vidya23002/technical-documentation-assistant>

Contents

Abstract	4
1 Introduction	5
2 Problem Statement	6
3 Objectives	7
4 Scope of the Project	8
5 Technologies Used	9
6 System Architecture	10
6.1 Architecture Diagram	11
7 Project Workflow	12
7.1 Document Loading	12
7.2 Text Chunking	12
7.3 Embedding Generation	12
7.4 Vector Database	13
7.5 Question Processing	13
7.6 Semantic Retrieval	13
7.7 Prompt Construction	13
7.8 Answer Generation	13
8 Important RAG Concepts	14
8.1 Retrieval-Augmented Generation	14
8.2 Embeddings	14
8.3 Semantic Search	14
8.4 Grounding	15
8.5 Knowledge Base	15

9	Modules	16
9.1	Document Loader	16
9.2	Chunking Module	16
9.3	Embedding Module	16
9.4	Vector Database Module	16
9.5	Retrieval Module	16
9.6	Generation Module	17
9.7	User Interface Module	17
10	Implementation	18
10.1	Document Extraction	18
10.2	Chunk Creation	18
10.3	Embedding Model	18
10.4	ChromaDB	19
10.5	Retrieval	19
10.6	Generation	19
11	User Interface	20
12	Testing and Results	21
13	Screenshots	22
13.1	Main Application	22
13.2	Question and Generated Answer	23
14	Advantages	24
15	Limitations	25
16	How to Run the Project	26
16.1	Step 1: Clone the Repository	26
16.2	Step 2: Create a Virtual Environment	26
16.3	Step 3: Activate the Environment	26
16.4	Step 4: Install Dependencies	26
16.5	Step 5: Configure API Key	26
16.6	Step 6: Run the Application	27
17	Project Structure	28
18	GitHub Repository	29

19 Viva Questions and Answers	30
19.1 What is RAG?	30
19.2 Why did you use RAG?	30
19.3 What are embeddings?	30
19.4 What is semantic search?	30
19.5 Why did you use Sentence Transformers?	30
19.6 Why did you use ChromaDB?	30
19.7 What is grounding?	31
19.8 How does your complete project work?	31
19.9 How do you reduce answers from outside the documentation?	31
19.10What is the role of ChromaDB?	31
19.11What is the role of Gemini?	31
19.12Why is chunking required?	31
20 Future Enhancements	32
21 Conclusion	33
References	34

Abstract

The Technical Documentation Assistant is a Retrieval-Augmented Generation (RAG) based question-answering system designed to help users obtain information from technical documentation quickly and efficiently.

Technical documentation can contain a large amount of information, making it difficult for users to manually search for specific concepts. The proposed system addresses this problem by converting the documentation into searchable semantic representations and retrieving the most relevant sections for a user's question.

The system uses Python for implementation, Streamlit for the user interface, pypdf for extracting text from PDF documentation, Sentence Transformers for creating embeddings, ChromaDB for vector storage and retrieval, and Google Gemini for generating the final response.

The system follows a Retrieval-Augmented Generation workflow. First, the documentation is loaded and divided into smaller chunks. These chunks are converted into embeddings and stored in ChromaDB. When the user enters a question, the question is also converted into an embedding. The system then retrieves the three most relevant documentation chunks. These chunks are provided to the Gemini model as context, and the model generates the final answer.

The application is designed to keep its answers grounded in the supplied documentation. If the required information is not available in the provided documentation, the system is instructed to inform the user rather than rely on outside information.

1. Introduction

The Technical Documentation Assistant is a Retrieval-Augmented Generation (RAG) based chatbot that allows users to ask questions about technical documentation.

Instead of requiring the user to manually search through a lengthy document, the system automatically retrieves relevant information and generates a natural-language response.

The project demonstrates the integration of document processing, embeddings, semantic search, vector databases, and generative artificial intelligence within a single application.

The current prototype uses Python Tutorial documentation as its knowledge base. The extracted documentation is processed and stored in a vector database. User questions are then matched with relevant documentation using semantic similarity.

2. Problem Statement

Technical documentation is often lengthy and contains information distributed across many pages and sections. Users may find it difficult to locate the exact information required for a particular question.

Traditional keyword-based search may also fail when the wording of the user's question differs from the wording used in the documentation.

Therefore, the project aims to develop a chatbot that can:

- Accept natural-language questions from users.
- Search technical documentation semantically.
- Retrieve relevant documentation sections.
- Generate understandable answers from the retrieved information.
- Reduce responses based on information outside the supplied knowledge base.

3. Objectives

The main objectives of the project are:

1. To create a searchable technical-documentation knowledge base.
2. To extract text from PDF documentation.
3. To divide large documentation into smaller manageable chunks.
4. To convert text into numerical embeddings.
5. To perform semantic retrieval using embeddings.
6. To store and retrieve embeddings using ChromaDB.
7. To provide retrieved documentation as context to a generative model.
8. To generate natural-language answers using Google Gemini.
9. To provide a simple web interface using Streamlit.
10. To display the retrieved documentation to improve transparency.

4. Scope of the Project

The current project focuses on answering questions from the supplied Python technical documentation.

The scope includes:

- PDF document processing.
- Text extraction.
- Text chunking.
- Embedding generation.
- Semantic retrieval.
- Vector database storage.
- Generative answer generation.
- Web-based interaction through Streamlit.

The prototype can be extended in the future to support multiple technical documentation sources such as programming languages, software libraries, frameworks, and APIs.

5. Technologies Used

Technology	Purpose
Python	Main programming language used to implement the application.
Streamlit	Used to create the web-based user interface.
pypdf	Used to extract text from the PDF documentation.
Sentence Transformers	Used to generate numerical embeddings for documentation and questions.
all-MiniLM-L6-v2	Embedding model used for semantic representation of text.
ChromaDB	Used for storing and retrieving document embeddings.
Google Gemini	Generates the final natural-language response using retrieved context.
GitHub	Used for source-code version control and project hosting.
VS Code	Development environment used for writing and testing the application.

6. System Architecture

The application follows a Retrieval-Augmented Generation pipeline.

6.1 Architecture Diagram

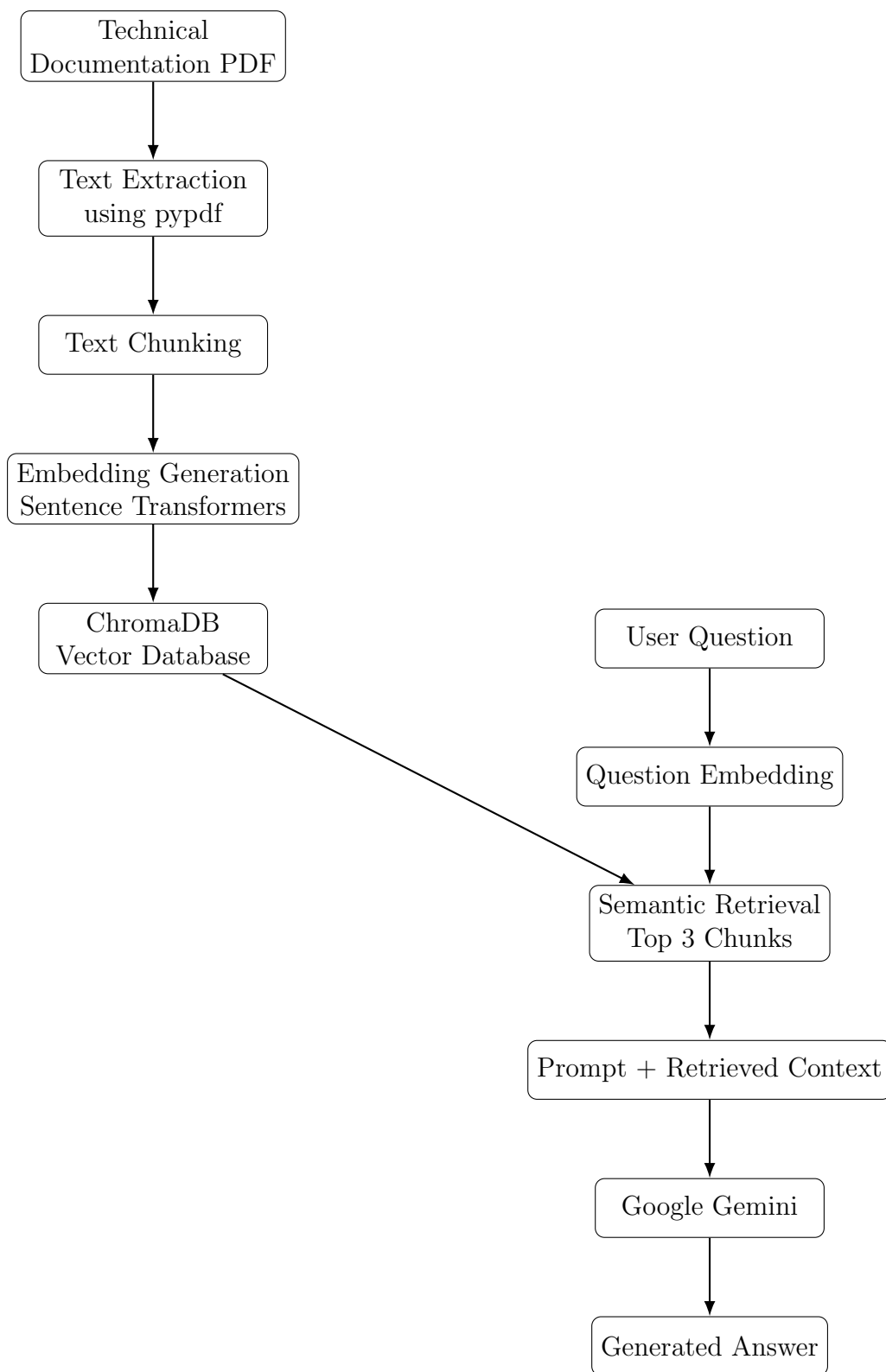


Figure 6.1: Architecture of the Technical Documentation Assistant

7. Project Workflow

7.1 Document Loading

The application reads the technical documentation PDF using the pypdf library.

The text from the PDF pages is extracted and combined into a single text collection.

The documentation file used in the prototype is:

Python Tutorial, Release 3.7.0

7.2 Text Chunking

Large documents cannot always be processed efficiently as one large piece of text.

Therefore, the extracted text is divided into smaller chunks.

The current implementation uses:

- Chunk size: 1000 characters
- Chunk overlap: 200 characters

The overlap helps preserve context between neighboring chunks.

7.3 Embedding Generation

After chunking, each documentation chunk is converted into an embedding.

The project uses the Sentence Transformers model:

all-MiniLM-L6-v2

An embedding is a numerical representation of text that can be used to measure semantic similarity.

7.4 Vector Database

The generated embeddings are stored in ChromaDB.

ChromaDB allows the application to efficiently retrieve documentation chunks that are semantically related to a user's question.

7.5 Question Processing

When the user enters a question, the same embedding model is used to convert the question into an embedding.

The question embedding is then compared with the stored documentation embeddings.

7.6 Semantic Retrieval

The system retrieves the three most relevant documentation chunks.

This allows the system to provide the generative model with focused and relevant context instead of the entire documentation.

7.7 Prompt Construction

The retrieved documentation chunks are combined into a context.

The context and the user's question are then included in a prompt sent to the Gemini model.

The prompt instructs the model to:

- Use only the supplied documentation.
- Answer the user's question clearly.
- Avoid using outside knowledge.
- State when the required information cannot be found.

7.8 Answer Generation

Google Gemini receives the question together with the retrieved documentation and generates a natural-language response.

The response is then displayed in the Streamlit application.

8. Important RAG Concepts

8.1 Retrieval-Augmented Generation

RAG stands for Retrieval-Augmented Generation.

It combines two major processes:

1. Retrieval of relevant information.
2. Generation of an answer using the retrieved information.

In this project:

Question $\rightarrow$ *Retrieval* $\rightarrow$ *Relevant Documentation* $\rightarrow$ *Generation* $\rightarrow$ *Answer*

8.2 Embeddings

An embedding is a numerical representation of text.

Texts with related meanings can have similar vector representations.

The project uses embeddings to find documentation that is semantically related to the user's question.

8.3 Semantic Search

Semantic search retrieves information according to meaning rather than depending only on exact keyword matches.

For example, a user may ask:

How can one class acquire properties and behavior from another class?

The documentation may use different wording, but semantic retrieval can still identify relevant content about inheritance.

8.4 Grounding

Grounding means generating a response based on supplied evidence or context.

In this project, the retrieved documentation is supplied to Gemini as context.

8.5 Knowledge Base

The knowledge base is the collection of information used by the retrieval system.

For this project, the knowledge base is the supplied Python documentation PDF.

9. Modules

9.1 Document Loader

Responsible for:

- Opening the PDF.
- Reading individual pages.
- Extracting text.
- Combining the extracted text.

9.2 Chunking Module

Responsible for dividing the extracted documentation into smaller overlapping chunks.

9.3 Embedding Module

Responsible for converting documentation chunks and user questions into embeddings using Sentence Transformers.

9.4 Vector Database Module

Responsible for storing embeddings and documentation chunks using ChromaDB.

9.5 Retrieval Module

Responsible for retrieving the three most relevant documentation chunks for the user's question.

9.6 Generation Module

Responsible for constructing the prompt and generating the final answer using Google Gemini.

9.7 User Interface Module

The Streamlit interface provides:

- Project title.
- Question input.
- Answer display.
- Retrieved documentation display.

10. Implementation

The main implementation is contained in the `app.py` file.

The application loads the PDF, creates chunks, loads the embedding model, creates the ChromaDB collection, and processes user questions.

10.1 Document Extraction

A simplified representation of the extraction process is:

```
reader = PdfReader(pdf_path)

all_text = ""

for page in reader.pages:
    text = page.extract_text()

    if text:
        all_text += text + "\n"
```

10.2 Chunk Creation

The project creates chunks using a chunk size and overlap:

```
chunk_size = 1000
overlap = 200
```

10.3 Embedding Model

The embedding model is loaded using:

```
SentenceTransformer("all-MiniLM-L6-v2")
```

10.4 ChromaDB

The vector database is initialized using:

```
chromadb.PersistentClient(  
    path="chroma_db"  
)
```

10.5 Retrieval

The application retrieves three relevant documents:

```
results = collection.query(  
    query_embeddings=[  
        question_embedding.tolist()  
    ],  
    n_results=3  
)
```

10.6 Generation

The retrieved context is passed to Gemini:

```
response = client.models.generate_content(  
    model="gemini-3.6-flash",  
    contents=prompt  
)
```

11. User Interface

The application is implemented using Streamlit.

The interface contains a text input field where users can enter technical questions.

After the question is submitted, the system:

1. Converts the question into an embedding.
2. Searches ChromaDB.
3. Retrieves relevant documentation.
4. Sends the context to Gemini.
5. Displays the generated answer.

12. Testing and Results

Testing is performed using questions related to the supplied technical documentation.

ID	Test Input	Expected Result
TC01	What is inheritance in Python?	Relevant documentation is retrieved and an answer is generated.
TC02	What is a function in Python?	Relevant documentation is retrieved and an answer is generated.
TC03	What are lists in Python?	Relevant documentation is retrieved and an answer is generated.
TC04	What is the history of Java?	If the information is not available in the supplied documentation, the system should state that it could not find the information in the provided documentation.

13. Screenshots

The screenshots below demonstrate the working application.

13.1 Main Application

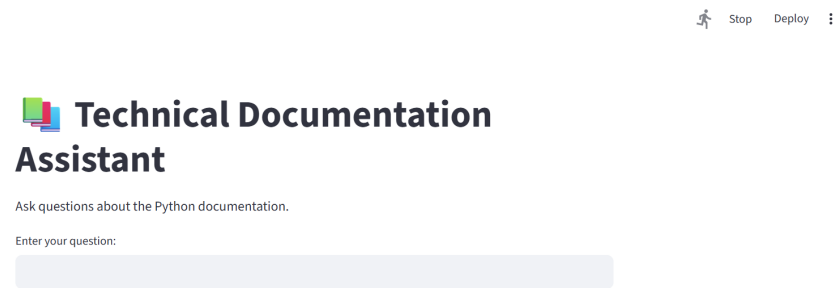
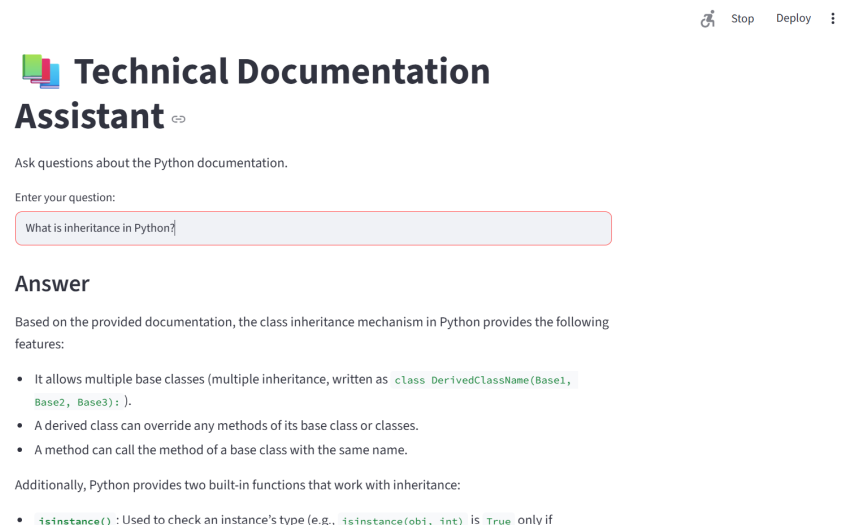


Figure 13.1: Main page of the Technical Documentation Assistant

13.2 Question and Generated Answer



The screenshot displays the 'Technical Documentation Assistant' interface. At the top right, there are icons for a document, 'Stop', 'Deploy', and a menu. The main heading is 'Technical Documentation Assistant' with a small icon to the left. Below this, a prompt says 'Ask questions about the Python documentation.' followed by 'Enter your question:'. A text input field contains the question 'What is inheritance in Python?'. Below the input field, the section 'Answer' is shown. The answer text states: 'Based on the provided documentation, the class inheritance mechanism in Python provides the following features:'. This is followed by a bulleted list:

- It allows multiple base classes (multiple inheritance, written as `class DerivedClassName(Base1, Base2, Base3):`).
- A derived class can override any methods of its base class or classes.
- A method can call the method of a base class with the same name.

Then, it says 'Additionally, Python provides two built-in functions that work with inheritance:'. This is followed by another bulleted list:

- `isinstance()` : Used to check an instance's type (e.g., `isinstance(obj, int)` is `True` only if

Figure 13.2: Question and generated answer

14. Advantages

1. Makes lengthy technical documentation easier to query.
2. Supports natural-language questions.
3. Uses semantic retrieval.
4. Provides documentation-grounded responses.
5. Allows users to inspect retrieved documentation.
6. Provides a simple web interface.
7. Can be extended to other technical documentation.

15. Limitations

1. The quality of answers depends on the supplied documentation.
2. The current prototype uses one main documentation knowledge base.
3. Questions requiring information absent from the documentation may not be answered.
4. Retrieval quality depends on the embedding model and document chunking.
5. The generation component depends on the configured Gemini API.

16. How to Run the Project

16.1 Step 1: Clone the Repository

The project source code is available on GitHub:

<https://github.com/vidya23002/technical-documentation-assistant>

16.2 Step 2: Create a Virtual Environment

Create a Python virtual environment:

```
python -m venv venv
```

16.3 Step 3: Activate the Environment

On Windows PowerShell:

```
venv\Scripts\activate
```

16.4 Step 4: Install Dependencies

Install the required packages:

```
pip install -r requirements.txt
```

16.5 Step 5: Configure API Key

Create:

```
.streamlit/secrets.toml
```

Add the Gemini API key locally.

The API key must not be uploaded to GitHub.

16.6 Step 6: Run the Application

Use:

```
python -m streamlit run app.py
```

The Streamlit application will provide a local URL that can be opened in a web browser.

17. Project Structure

The project is organized approximately as follows:

```
technical-documentation-assistant/  
|  
|-- app.py  
|-- requirements.txt  
|-- .gitignore  
|  
|-- documents/  
| '-- python_documentation.pdf  
|  
|-- screenshots/  
|  
|-- .streamlit/  
| '-- secrets.toml  
|  
'-- chroma_db/
```

The `secrets.toml`, virtual environments, and local ChromaDB files should not be uploaded to GitHub.

18. GitHub Repository

The complete project source code is available at:

<https://github.com/vidya23002/technical-documentation-assistant>

The repository contains the main application code, documentation PDF, requirements file, screenshots, and project configuration.

19. Viva Questions and Answers

19.1 What is RAG?

RAG stands for Retrieval-Augmented Generation. It retrieves relevant information from a knowledge base and provides that information to a generative model as context before generating an answer.

19.2 Why did you use RAG?

RAG is used to make the chatbot answer questions using the supplied technical documentation instead of depending only on general model knowledge.

19.3 What are embeddings?

Embeddings are numerical vector representations of text. They allow the system to compare the semantic similarity between a question and document chunks.

19.4 What is semantic search?

Semantic search retrieves information based on meaning rather than depending only on exact keyword matching.

19.5 Why did you use Sentence Transformers?

Sentence Transformers provides a practical way to convert text into embeddings that can be used for semantic similarity and retrieval.

19.6 Why did you use ChromaDB?

ChromaDB is used to store document embeddings and retrieve relevant documents for a user's question.

19.7 What is grounding?

Grounding means keeping a generated response tied to supplied evidence or context. In this project, retrieved documentation is supplied as context to the generative model.

19.8 How does your complete project work?

The complete workflow is:

PDF → Text Extraction → Chunking → Embeddings → ChromaDB → Question
Embedding → Retrieval → Prompt → Gemini → Answer

19.9 How do you reduce answers from outside the documentation?

The generation prompt instructs Gemini to use only the provided documentation. If the required information is not found, the model is instructed to state that the information could not be found in the provided documentation.

19.10 What is the role of ChromaDB?

ChromaDB acts as the vector database in the project. It stores the documentation chunks and their embeddings and retrieves relevant chunks for new questions.

19.11 What is the role of Gemini?

Gemini is the generative component. It receives the user's question together with the retrieved documentation context and generates the final natural- language response.

19.12 Why is chunking required?

Chunking divides a large document into smaller pieces that can be embedded and retrieved effectively. Overlapping chunks also help preserve context between neighboring pieces.

20. Future Enhancements

The project can be extended in several ways:

- Support multiple technical documentation files.
- Add document upload functionality.
- Add metadata such as page numbers and section names.
- Add citation links to the exact documentation sections.
- Add reranking to improve retrieval relevance.
- Support multiple programming languages.
- Deploy the application online.
- Add conversation history.
- Improve chunking strategies.

21. Conclusion

The Technical Documentation Assistant demonstrates the practical use of Retrieval-Augmented Generation for technical question answering.

The project combines PDF processing, text chunking, embeddings, semantic retrieval, vector database storage, and generative AI in a single application.

The system retrieves relevant documentation using ChromaDB and provides the retrieved content to Google Gemini as context. The Streamlit interface makes the system easy to use and allows users to inspect the retrieved documentation.

Overall, the project demonstrates how RAG can be used to make technical documentation easier to search and understand while keeping generated responses grounded in the supplied knowledge base.

References

1. Python Documentation / Python Tutorial.
2. Sentence Transformers documentation.
3. ChromaDB documentation.
4. Streamlit documentation.
5. Google Gemini API documentation.
6. GitHub repository: <https://github.com/vidya23002/technical-documentation-assistant>